

File permissions in Linux

Project description

At my organization, the research team needed me to update file permissions for some files and directories in the `projects` directory. The existing permissions didn't match the level of access people should actually have, so I went through and checked everything, then fixed what needed fixing.

Check file and directory details

First, I needed to see what permissions were already set. I ran `ls -la` in the `projects` directory — the `-la` flags give a detailed listing and also show hidden files, which is important since one of the files here turned out to be hidden.

```
researcher2@5d738f0f927b:~/projects$ ls -la
total 32
drwxr-xr-x 3 researcher2 research_team 4096 Dec  2 15:27 .
drwxr-xr-x 3 researcher2 research_team 4096 Dec  2 15:27 ..
-rw--w--- 1 researcher2 research_team  46 Dec  2 15:27 .project_x.txt
drwx--x--- 2 researcher2 research_team 4096 Dec  2 15:27 drafts
-rw-rw-rw- 1 researcher2 research_team  46 Dec  2 15:27 project_k.txt
-rw-r----- 1 researcher2 research_team  46 Dec  2 15:27 project_m.txt
-rw-rw-r-- 1 researcher2 research_team  46 Dec  2 15:27 project_r.txt
-rw-rw-r-- 1 researcher2 research_team  46 Dec  2 15:27 project_t.txt
researcher2@5d738f0f927b:~/projects$
```

This showed one directory (`drafts`), one hidden file (`.project_x.txt`), and five other project files. The 10-character string at the start of each line is what tells you the permissions.

Describe the permissions string

Each file's permissions break down into a 10-character string. The first character tells you whether it's a directory (`d`) or a regular file (`-`). After that, it's three groups of three characters each, covering read (`r`), write (`w`), and execute (`x`) — first for the user, then the group, then everyone else ("other"). A hyphen just means that permission isn't granted.

Change file permissions

The organization wanted to make sure others didn't have write access to any files. Looking back at my `ls -la` output, `project_k.txt` stood out — others had write access (`rw-`) when it shouldn't have. I fixed that with:

```
researcher2@5d738f0f927b:~/projects$ chmod o-w project_k.txt
researcher2@5d738f0f927b:~/projects$ ls -la
total 32
drwxr-xr-x 3 researcher2 research_team 4096 Dec  2 15:27 .
drwxr-xr-x 3 researcher2 research_team 4096 Dec  2 15:27 ..
-rw--w--- 1 researcher2 research_team  46 Dec  2 15:27 .project_x.txt
drwx--x--- 2 researcher2 research_team 4096 Dec  2 15:27 drafts
-rw-rw-r-- 1 researcher2 research_team  46 Dec  2 15:27 project_k.txt
-rw-r----- 1 researcher2 research_team  46 Dec  2 15:27 project_m.txt
-rw-rw-r-- 1 researcher2 research_team  46 Dec  2 15:27 project_r.txt
-rw-rw-r-- 1 researcher2 research_team  46 Dec  2 15:27 project_t.txt
researcher2@5d738f0f927b:~/projects$
```

After running `chmod o-w project_k.txt`, I reran `ls -la` to confirm — others' write access was gone, leaving it at `rw-r--` for that file.

Change file permissions on a hidden file

Next was `.project_x.txt`, which had recently been archived. The team didn't want anyone to have write access to it anymore, but they did want the group to be able to read it. I used:

```
researcher2@3213bbc1d047:~/projects$ chmod u-w,g-w,g+r .project_x.txt
researcher2@3213bbc1d047:~/projects$ ls -la
total 32
drwxr-xr-x 3 researcher2 research_team 4096 Dec 20 15:36 .
drwxr-xr-x 3 researcher2 research_team 4096 Dec 20 15:36 ..
-r--r----- 1 researcher2 research_team  46 Dec 20 15:36 .project_x.txt
drwx--x--- 2 researcher2 research_team 4096 Dec 20 15:36 drafts
-rw-rw-rw- 1 researcher2 research_team  46 Dec 20 15:36 project_k.txt
-rw-r----- 1 researcher2 research_team  46 Dec 20 15:36 project_m.txt
-rw-rw-r-- 1 researcher2 research_team  46 Dec 20 15:36 project_r.txt
-rw-rw-r-- 1 researcher2 research_team  46 Dec 20 15:36 project_t.txt
researcher2@3213bbc1d047:~/projects$
```

That one command handled all three changes at once: `u-w` took write away from the user, `g-w` took it away from the group, and `g+r` gave the group read access. The result was `r--r-----` — read-only for user and group, nothing for other.

Change directory permissions

Last was the `drafts` directory. The organization only wanted `researcher2` to be able to access it — meaning no one else should have execute permissions on it. Checking back, the group still had execute access, so I removed it:

```
researcher2@5d738f0f927b:~/projects$ chmod g-x drafts
researcher2@5d738f0f927b:~/projects$ ls -la
total 32
drwxr-xr-x 3 researcher2 research_team 4096 Dec  2 15:27 .
drwxr-xr-x 3 researcher2 research_team 4096 Dec  2 15:27 ..
-r--r----- 1 researcher2 research_team  46 Dec  2 15:27 .project_x.txt
drwx----- 2 researcher2 research_team 4096 Dec  2 15:27 drafts
-rw-rw-r-- 1 researcher2 research_team  46 Dec  2 15:27 project_k.txt
-rw-r----- 1 researcher2 research_team  46 Dec  2 15:27 project_m.txt
-rw-rw-r-- 1 researcher2 research_team  46 Dec  2 15:27 project_r.txt
-rw-rw-r-- 1 researcher2 research_team  46 Dec  2 15:27 project_t.txt
researcher2@5d738f0f927b:~/projects$
```

`researcher2` already had execute access, so I didn't need to add anything there — just `chmod g-x drafts` to take it away from the group. Now `drafts` show `drwx-----`, meaning only the owner can get into it.

Summary

Overall, I used `ls -la` to see where the existing permissions didn't match what the organization wanted, then went through and fixed each one with `chmod` — removing write access where it shouldn't exist, adding read access where it was needed, and locking down the `drafts` directory so only `researcher2` could access it.